

DATA STRUCTURES AND ALGORITHMS

LECTURE 7

Lect. PhD. Oneț-Marian Zsuzsanna

Babeș - Bolyai University
Computer Science and Mathematics Faculty

2025 - 2026

- XOR list
- Skip list
- Singly Linked Lists on Array

Today

- Doubly linked list on array
- Iterator
- Stack

SLL on Array - recap

- It is a linked list, but the elements are stored in an array. Each element has a *link*, denoting the next element, but this *link* is not a pointer, it is the position of the next element in the array.

elems		78	11	6	59	19		44		
next	7	6	5	-1	8	4	9	2	10	-1

head = 3

firstEmpty = 1

SLL on Array - Representation

- The representation of a singly linked list on an array is the following:

SLLA:

```
elems: TElem[]  
next: Integer[]  
cap: Integer  
head: Integer  
firstEmpty: Integer
```

SLLA - DeleteElement

subalgorithm deleteElement(slla, elem) **is:**

//pre: slla is a SLLA; elem is a TElem

//post: the element elem is deleted from SLLA

nodC $\leftarrow$ slla.head

prevNode $\leftarrow$ -1

while nodC $\neq$ -1 **and** slla.elems[nodC] $\neq$ elem **execute**

 prevNode $\leftarrow$ nodC

 nodC $\leftarrow$ slla.next[nodC]

end-while

if nodC $\neq$ -1 **then**

if nodC = slla.head **then**

 slla.head $\leftarrow$ slla.next[slla.head]

else

 slla.next[prevNode] $\leftarrow$ slla.next[nodC]

end-if

//continued on the next slide...

SLLA - DeleteElement

```
//add the nodC position to the list of empty spaces  
slla.next[nodC]  $\leftarrow$  slla.firstEmpty  
slla.firstEmpty  $\leftarrow$  nodC  
else  
  @the element does not exist  
end-if  
end-subalgorithm
```

- Complexity: $O(n)$

- How would you define an iterator for an SLLA?

- How would you define an iterator for an SLLA?
- Iterator for a SLLA is a combination of an iterator for an array and of an iterator for a singly linked list:
- Since the elements are stored in an array, the *currentElement* will be an index from the array.
- But since we have a linked list, going to the next element will not be done by incrementing the *currentElement* by one; we have to follow the *next* links.
- Also, initialization will be done with the position of the head, not position 1.

- Obviously, we can define a doubly linked list as well without pointers, using arrays.
- For the DLLA we will see another way of representing a linked list on arrays:
 - The main idea is the same, we will use array indexes as links between elements
 - We are using the same information, but we are going to structure it differently
 - However, we can make it look more similar to linked lists with dynamic allocation

- Linked Lists with dynamic allocation are made of nodes. We can define a structure to represent a node, even if we are working with arrays.
- A node (for a doubly linked list) contains the information and links towards the previous and the next nodes:

DLLANode:

info: TElem
next: Integer
prev: Integer

- Having defined the *DLLANode* structure, we only need one array, which will contain *DLLANodes*.
- Since it is a doubly linked list, we keep both the head and the tail of the list.

DLLA:

nodes: DLLANode[]

cap: Integer

head: Integer

tail: Integer

firstEmpty: Integer

size: Integer *//it is not mandatory, but useful*

DLLA - Allocate and free

- To make the representation and implementation even more similar to a dynamically allocated linked list, we can define the *allocate* and *free* functions as well.

function allocate(dlla) **is:**

//pre: dlla is a DLLA

//post: a new element will be allocated and its position returned

newElem $\leftarrow$ dlla.firstEmpty

if newElem $\neq$ -1 **then**

 dlla.firstEmpty $\leftarrow$ dlla.nodes[dlla.firstEmpty].next

if dlla.firstEmpty $\neq$ -1 **then**

 dlla.nodes[dlla.firstEmpty].prev $\leftarrow$ -1

end-if

 dlla.nodes[newElem].next $\leftarrow$ -1

 dlla.nodes[newElem].prev $\leftarrow$ -1

end-if

 allocate $\leftarrow$ newElem

end-function

DLLA - Allocate and free

subalgorithm free (dlla, poz) **is:**

//pre: dlla is a DLLA, poz is an integer number

//post: the position poz was freed

`dlla.nodes[poz].next  $\leftarrow$  dlla.firstEmpty`

`dlla.nodes[poz].prev  $\leftarrow$  -1`

if `dlla.firstEmpty  $\neq$  -1` **then**

`dlla.nodes[dlla.firstEmpty].prev  $\leftarrow$  poz`

end-if

`dlla.firstEmpty  $\leftarrow$  poz`

end-subalgorithm

DLLA - InsertPosition

subalgorithm insertPosition(dlla, elem, poz) **is:**

//pre: dlla is a DLLA, elem is a TElem, poz is an integer number

//post: the element elem is inserted in dlla at position poz

if $\text{poz} < 1$ **OR** $\text{poz} > \text{dlla.size} + 1$ **execute**

 @throw exception

end-if

$\text{newElem} \leftarrow \text{allocate}(\text{dlla})$

if $\text{newElem} = -1$ **then**

 @resize

$\text{newElem} \leftarrow \text{allocate}(\text{dlla})$

end-if

$\text{dlla.nodes}[\text{newElem}].\text{info} \leftarrow \text{elem}$

if $\text{poz} = 1$ **then**

if $\text{dlla.head} = -1$ **then**

$\text{dlla.head} \leftarrow \text{newElem}$

$\text{dlla.tail} \leftarrow \text{newElem}$

else

//continued on the next slide...

DLLA - InsertPosition

```
dlla.nodes[newElem].next  $\leftarrow$  dlla.head  
dlla.nodes[dlla.head].prev  $\leftarrow$  newElem  
dlla.head  $\leftarrow$  newElem
```

end-if

else

```
nodC  $\leftarrow$  dlla.head
```

```
pozC  $\leftarrow$  1
```

while $\text{nodC} \neq -1$ **and** $\text{pozC} < \text{poz} - 1$ **execute**

```
    nodC  $\leftarrow$  dlla.nodes[nodC].next
```

```
    pozC  $\leftarrow$  pozC + 1
```

end-while

if $\text{nodC} \neq -1$ **then** *//it should never be -1, the position is correct*

```
    nodNext  $\leftarrow$  dlla.nodes[nodC].next
```

```
    dlla.nodes[newElem].next  $\leftarrow$  nodNext
```

```
    dlla.nodes[newElem].prev  $\leftarrow$  nodC
```

```
    dlla.nodes[nodC].next  $\leftarrow$  newElem
```

//continued on the next slide...

DLLA - InsertPosition

```
    if nodNext = -1 then
        dlla.tail ← newElem
    else
        dlla.nodes[nodNext].prev ← newElem
    end-if
end-if
end-subalgorithm
```

- Complexity: $O(n)$

- The iterator for a DLLA contains as *current element* the index of the current node from the array.

DLLAIterator:

list: DLLA

currentElement: Integer

subalgorithm init(it, dlla) **is:**

//pre: dlla is a DLLA

//post: it is a DLLAlterator for dlla

it.list $\leftarrow$ dlla

it.currentElement $\leftarrow$ dlla.head

end-subalgorithm

- For a (dynamic) array, currentElement is set to 1 when an iterator is created. For a DLLA we need to set it to the head of the list (which might be position 1, but it might be a different position as well).
- Complexity:

subalgorithm init(it, dlla) **is:**

//pre: dlla is a DLLA

//post: it is a DLLAlterator for dlla

it.list $\leftarrow$ dlla

it.currentElement $\leftarrow$ dlla.head

end-subalgorithm

- For a (dynamic) array, currentElement is set to 1 when an iterator is created. For a DLLA we need to set it to the head of the list (which might be position 1, but it might be a different position as well).
- Complexity: $\Theta(1)$

DLLAlterator - getCurrent

subalgorithm getCurrent(it) **is:**

//pre: it is a DLLAlterator, it is valid

//post: e is a TElem, e is the current element from it

//throws exception if the iterator is not valid

if it.currentElement = -1 **then**

 @throw exception

end-if

getCurrent $\leftarrow$ it.list.nodes[it.currentElement].info

end-subalgorithm

- Complexity:

DLLAlterator - getCurrent

subalgorithm getCurrent(it) **is:**

//pre: it is a DLLAlterator, it is valid

//post: e is a TElem, e is the current element from it

//throws exception if the iterator is not valid

if it.currentElement = -1 **then**

 @throw exception

end-if

getCurrent $\leftarrow$ it.list.nodes[it.currentElement].info

end-subalgorithm

- Complexity: $\Theta(1)$

subalgorithm next (it) **is:**

//pre: it is a DLLAlterator, it is valid

//post: the current elements from it is moved to the next element

//throws exception if the iterator is not valid

if it.currentElement = -1 **then**

 @throw exception

end-if

it.currentElement $\leftarrow$ it.list.nodes[it.currentElement].next

end-subalgorithm

- In case of a (dynamic) array, going to the next element means incrementing the *currentElement* by one. For a DLLA we need to follow the links.
- Complexity:

subalgorithm next (it) **is:**

//pre: it is a DLLAlterator, it is valid

//post: the current elements from it is moved to the next element

//throws exception if the iterator is not valid

if it.currentElement = -1 **then**

 @throw exception

end-if

it.currentElement $\leftarrow$ it.list.nodes[it.currentElement].next

end-subalgorithm

- In case of a (dynamic) array, going to the next element means incrementing the *currentElement* by one. For a DLLA we need to follow the links.
- Complexity: $\Theta(1)$

function valid (it) **is:**

//pre: it is a DLLAlterator

//post: valid return true is the current element is valid, false otherwise

if it.currentElement = -1 **then**

 valid $\leftarrow$ False

else

 valid $\leftarrow$ True

end-if

end-function

- Complexity:

function valid (it) **is:**

//pre: it is a DLLAlterator

//post: valid return true is the current element is valid, false otherwise

if it.currentElement = -1 **then**

 valid $\leftarrow$ False

else

 valid $\leftarrow$ True

end-if

end-function

- Complexity: $\Theta(1)$

Iterator - why do we need it? I

- Most containers have iterators and for (almost) every data structure we will discuss how we can implement an iterator for a container defined on that data structure.
- Why are iterators so important?

Iterator - why do we need it? II

- They offer a uniform way of iterating through the elements of any container

subalgorithm printContainer(c) **is:**

//pre: c is a container

//post: the elements of c were printed

//we create an iterator using the iterator method of the container

iterator(c, it)

while valid(it) **execute**

//get the current element from the iterator

elem ← getCurrent(it)

print elem

//go to the next element

next(it)

end-while

end-subalgorithm

Iterator - why do we need it? III

- For most containers the iterator is the only thing we have that lets us see the content of the container.
 - ADT List is the only container that has positions, for other containers we can use only the iterator.

Iterator - why do we need it? IV

- Giving up positions, we can gain performance.
 - Containers that do not have positions can be represented on data structures where some operations have good complexities, but where the notion of a position does not naturally exist and where enforcing positions is really complicated.

Iterator - why do we need it? V

- Even if we have positions, using an iterator might be faster.
 - Going through the elements of a linked list with an iterator is faster than going through every position one-by-one.

- The ADT *Stack* represents a container in which access to the elements is restricted to one end of the container, called the *top* of the stack.
 - When a new element is added, it will automatically be added to the top.
 - When an element is removed, the one from the top is automatically removed.
 - Only the element from the top can be accessed.
- Because of this restricted access, the stack is said to have a **LIFO** policy: **L**ast **I**n, **F**irst **O**ut (the last element that was added will be the first element that will be removed).

Representation for Stack

- Data structures that can be used to implement a stack:
 - Arrays
 - Static Array - if we want a fixed-capacity stack
 - Dynamic Array
 - Linked Lists
 - Singly-Linked List
 - Doubly-Linked List

Array-based representation

- Where should we place the top of the stack for optimal performance?

Array-based representation

- Where should we place the top of the stack for optimal performance?
- We have two options:
 - Place top at the beginning of the array - every push and pop operation needs to shift every element to the right or left.
 - Place top at the end of the array - push and pop elements without moving the other ones.
- Conclusion: put it at the end of the array

Stack - Representation on SLL

- Where should we place the top of the stack for optimal performance?

Stack - Representation on SLL

- Where should we place the top of the stack for optimal performance?
- We have two options:
 - Place it at the end of the list (like we did when we used an array) - for every push, pop and top operation we have to iterate through every element to get to the end of the list.
 - Place it at the beginning of the list - we can push and pop elements without iterating through the list.
- Conclusion: put it at the beginning of the SLL

Stack - Representation on DLL

- Where should we place the top of the stack for optimal performance?

Stack - Representation on DLL

- Where should we place the top of the stack for optimal performance?
- We have two options:
 - Place it at the end of the list (like we did when we used an array) - we can push and pop elements without iterating through the list.
 - Place it at the beginning of the list - we can push and pop elements without iterating through the list.
- Conclusion: you can put it at either end of the DLL

Fixed capacity stack with linked list

- How could we implement a stack with a fixed maximum capacity using a linked list?

Fixed capacity stack with linked list

- How could we implement a stack with a fixed maximum capacity using a linked list?
- Similar to the implementation with a static array, we can keep in the *Stack* structure two integer values (besides the top node): maximum capacity and current size.

GetMinimum in constant time

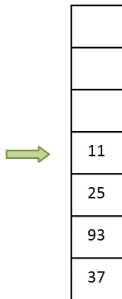
- How can we design a *special stack* that has a *getMinimum* operation with $\Theta(1)$ time complexity (and the other operations have $\Theta(1)$ time complexity as well)?

GetMinimum in constant time

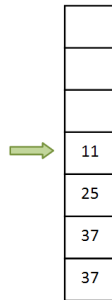
- How can we design a *special stack* that has a *getMinimum* operation with $\Theta(1)$ time complexity (and the other operations have $\Theta(1)$ time complexity as well)?
- We can keep an auxiliary stack, containing as many elements as the original stack, but containing the minimum value up to each element. Let's call this auxiliary stack a *min stack* and the original stack the *element stack*.

GetMinimum in constant time - Example

- If this is the *element stack*:



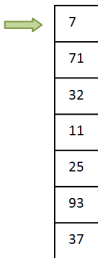
- This is the corresponding *min stack*:



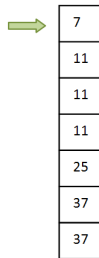
GetMinimum in constant time - Example

- When a new element is pushed to the *element stack*, we push a new element to the *min stack* as well. This element is the minimum between the top of the *min stack* and the newly added element.

- The *element stack*:



- The corresponding *min stack*:



GetMinimum in constant time

- When an element s_i is popped from the *element stack*, we will pop an element from the *min stack* as well.
- The *getMinimum* operation will simply return the *top* of the *min stack*.
- The other stack operations remain unchanged (except *init*, where you have to create two stacks).

GetMinimum in constant time

- Let's implement the *push* operation for this *SpecialStack*, represented in the following way:

SpecialStack:

elementStack: Stack

minStack: Stack

- We will use an existing implementation for the stack and work only with the operations from the interface.

Push for SpecialStack

```
subalgorithm push(ss, e) is:
  if isFull(ss.elementStack) then
    @throw overflow (full stack) exception
  end-if
  if isEmpty(ss.elementStack) then //the stacks are empty, just push the elem
    push(ss.elementStack, e)
    push(ss.minStack, e)
  else
    push(ss.elementStack, e)
    currentMin  $\leftarrow$  top(ss.minStack)
    if currentMin < e then //find the minim to push to minStack
      push(ss.minStack, currentMin)
    else
      push(ss.minStack, e)
    end-if
  end-if
end-subalgorithm //Complexity:  $\Theta(1)$ 
```


- We designed the special stack in such a way that all the operations have a $\Theta(1)$ time complexity.
- The disadvantage is that we occupy twice as much space as with the regular stack.
- Think about how can we reduce the space occupied by the *min stack* to $O(n)$ (especially if the minimum element of the stack rarely changes). *Hint: If the minimum does not change, we don't have to push a new element to the min stack.* How can we implement the *push* and *pop* operations in this case? What happens if the minimum element appears more than once in the *element stack*?

- Today we have talked about:
 - Doubly linked list on array
 - Why iterators are important
 - How to implement a Stack efficiently on different data structures.
- Extra reading: my solution for last week's problem statement.